Parallel Programming HW3

108062329 何品樺

1. **Implementation**
   a. **Which algorithm do you choose in hw3-1?**
   I used sequential template of blocked-Floyd-Warshall algorithm as base, added an OMP field on the calculation of blocks, parallels it along x-dimension.

```
void cal(
    int B, int Round, int block_start_x, int block_start_y, int block_width
    int block_end_x = block_start_x + block_height;
    int block_end_y = block_start_y + block_width;

    #pragma omp parallel for num_threads(16) schedule(dynamic)
    for (int b_i = block_start_x; b_i < block_end_x; ++b_i) {
        for (int b_j = block_start_y; b_j < block_end_y; ++b_j) {
            // To calculate B*B elements in the block (b_i, b_j)
            // For each block, it need to compute B times
            for (int k = Round * B; k < (Round + 1) * B && k < n; ++k) {
                // To calculate original index of elements in the block (b_
                // For instance, original index of (0,0) in block (1,2) is
                int block_internal_start_x = b_i * B;
```

   b. **How do you divide your data in hw3-2, hw3-3?**
   I divide the data into 64 by 64 blocks to fit under per-block-shared-memory size limit. The kernel function calculates one 64x64 block. In 3-3, I divide the data for 2 GPUs. In phase 1 I just let GPU 1 do the calculation; In phase 2, one GPU handles the pivot row and the other handles pivot column; Finally, in phase 3, I simply divide the whole matrix into two chunks, the upper(0~(round/2) rows), and the lower((round/2)+1~n rows), each belongs to one GPU.
   c. **What's your configuration in hw3-2, hw3-3? And why? (e.g. blocking factor, #blocks, #threads)**
   I choose blocking factor to be 64 to fit under per-block-shared-memory size limit. The number of GPU blocks for each kernel launch is the number of data block we want to calculate, each GPU block will be responsible for different data blocks. As for the number of threads, I use 64 * 16 (=1024) to maximize core usage.
   d. **How do you implement the communication in hw3-3?**
   I use zero-copy memory since it's easier to implement.

2. **Profiling Results**

The largest part is phase 3, I use data **p13k1** for profiling.

	Min	Max	Average	
Global Load	17.34	17.533	17.425	**GB/s**
Global Store	38.511	44.871	44.627	**GB/s**
Shared Load	4131.9	4153.9	4147	**GB/s**
Shared Store	89.725	89.962	89.84	**GB/s**
Occupancy	91.86%	92.56%	92.39%	
SM Efficiency	99.93%	99.97%	99.95%	

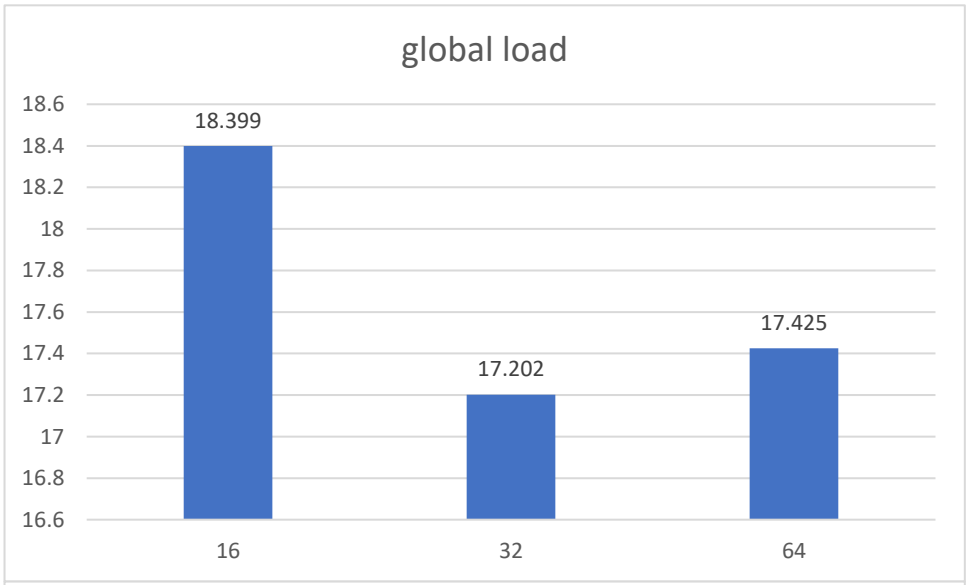
3. **Experiment & Analysis**

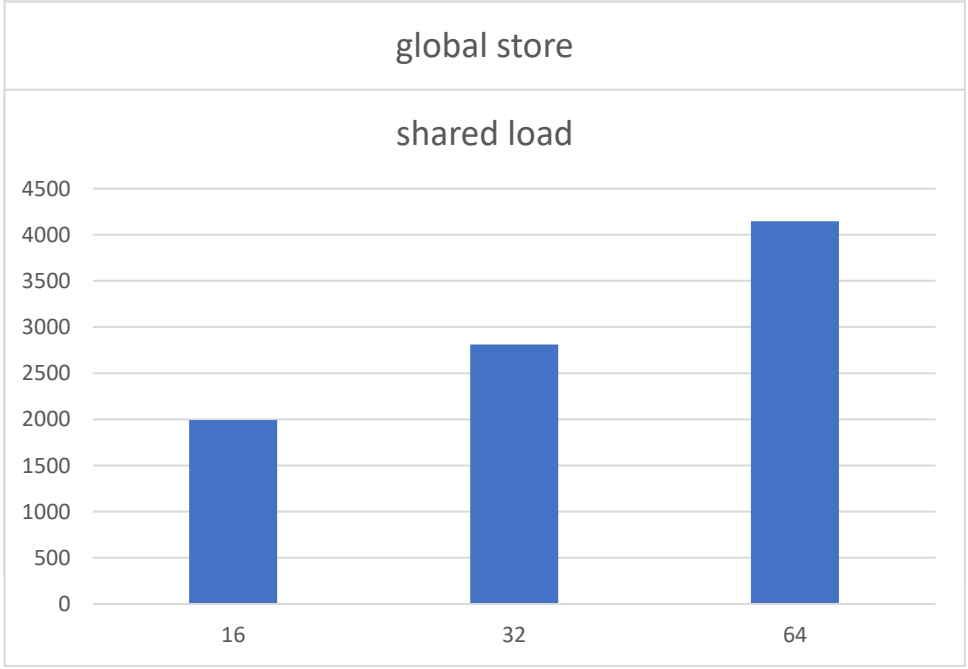
   a. **System Spec**

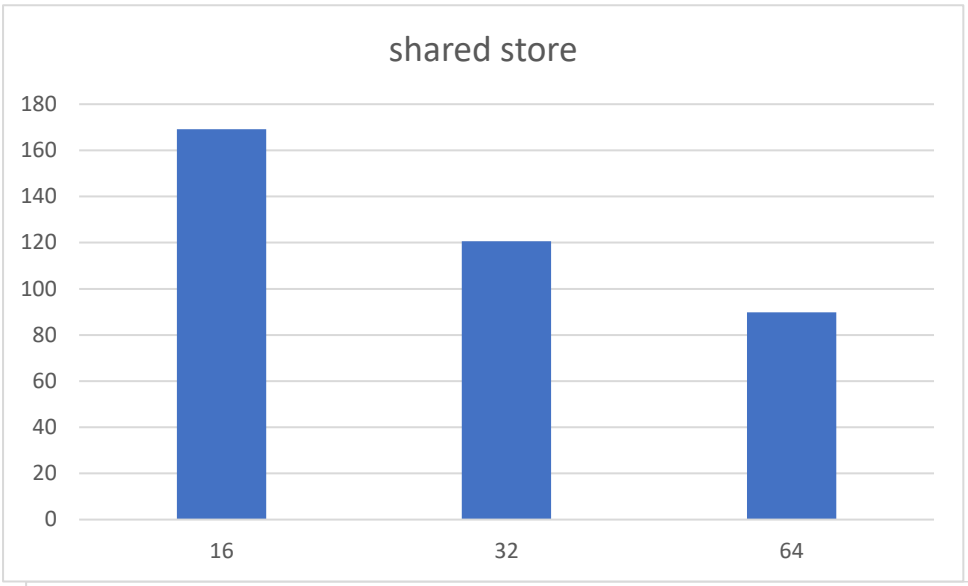
   Hades server

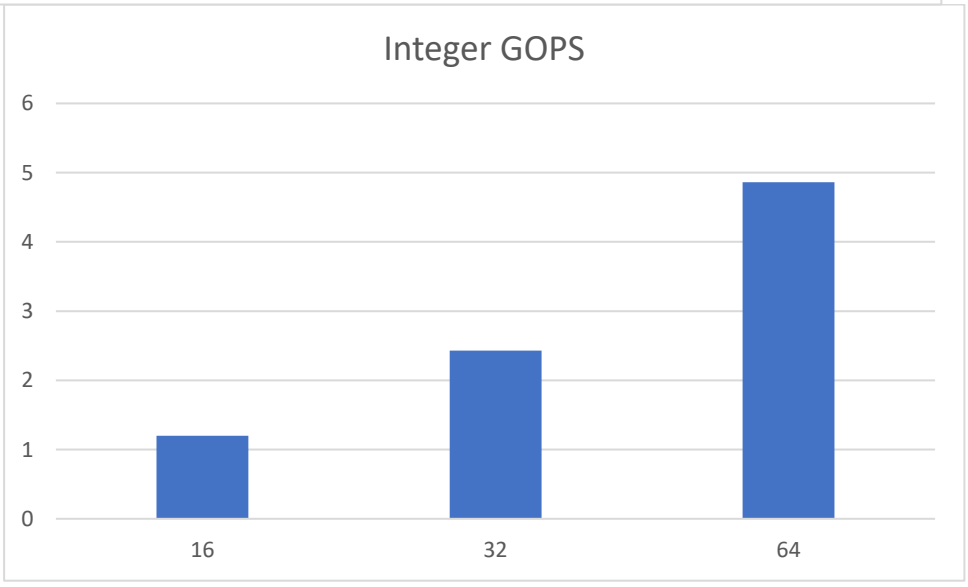
   b. **Blocking Factor**

   With shared memory, blocking factor can't exceed 64 in my implementation.









c. Optimization